

A CAPABILITY DELTA FOR THE FORGE PROJECT

Forge: Constructive Exactness

Integral homology, certified reductions,
and obstruction-aware homotopy synthesis

Prepared for Vladimir Reshetnikov

September 15, 2026

What is delivered. A self-contained algorithmic proposal, proofs of its certificate principles, a runnable exact-integer Python prototype, an independent arithmetic replayer, 656 recorded certificate cases, and adversarial and differential tests. The companion archive contains all source and evidence.

What is not delivered. An installed Lean tactic or a verified Lean checker. No Lean compiler was available in this authoring environment. The integration is specified, not presented as compiled code; no comparison against `grind` or another Lean tactic was run.

Audited Forge snapshot. `58ea206bd7ad501b930add568151217bcc7f82a2`. The contribution is relative to the reviewed capability register and merged architecture at this revision, not a claim of mathematical priority.

Abstract

Forge already has integer-lattice witnesses, invariant closures, finite quotients, noncommutative certificates, analytic workers, and quantitative reasoning. Another generic solver wrapper would largely duplicate that work. This article develops a different source fragment: finite free integral chain complexes, with *constructive exactness* as the common interface. The worker does not merely compute ranks. It constructs coordinates for every cycle modulo boundaries, produces explicit fillers or separating functionals, transports maps through quotient presentations, reduces complexes while keeping chain homotopies, and solves the coupled equations for a homotopy between two chain maps. A negative answer carries a dual matrix functional, including modular obstructions that cannot be seen by rational linear algebra.

The most revealing example consists of two maps that induce the same maps on all homology groups but are not integrally chain-homotopic: the obstruction is $2a + 2b = 1$. The key architectural improvement is to combine certified reduction with whole-diagram synthesis, then transport the resulting witness back to the original source objects. In a controlled family, this changes a 288-equation, 144-variable problem rejected by the prototype’s equation cap into an empty reduced problem whose lifted homotopy passes direct replay.

The supplied search uses SymPy 1.14.0. Its checker uses only Python’s standard library and does not run Smith reduction or reconstruct the homotopy equation assembler. All 656 stored certificates replay with site packages disabled; 489 invalid controls are rejected. Separate checks comprise 64 all-minors normal-form comparisons, 24 coordinate-composition identities, 184 tiny modular enumerations, and 12 dependency-free unit tests. These are different units, not a combined success rate. Classical mathematics is reused explicitly; the new contribution is a concrete Forge worker, its source obligations, certificate language, transport laws, and executable evidence.

Contents

1	The extension, and the boundary of the audit	3
1.1	Do not build another version of the existing proposal	3
1.2	What was inspected	3
1.3	A source fragment, not a competing foundation	4
2	A running example: a cycle whose double can be filled	4
3	Integral coordinate certificates	5
3.1	The exact input and the normal-form receipt	5
3.2	First transformation: the entire integral kernel	5
3.3	Second transformation: boundaries inside the kernel	6
3.4	A complete filler-or-obstruction algorithm	7
3.5	Universal exactness, class orders, and useful asymmetries	7
3.6	Complexity and certificate size	8
4	Maps on homology: preserve relations, not just group names	8
5	Reduction with retained chain homotopies	9
5.1	Why an invariant alone is not a sufficient reduction result	9
5.2	Single-pair cancellation over the integers	10
5.3	Composition without retaining the search history	11
5.4	Pivot policy, stopping, and a deliberate failure of the heuristic	11
5.5	What reduction does not establish	12
6	Whole-diagram homotopy synthesis and dual obstruction	12
6.1	The synthesis problem	12
6.2	A concrete integer system	13

6.3	Negative answers that can be checked without the assembler	13
6.4	Equal homology maps are not enough	14
7	Reduction-first synthesis with certified lifting	15
7.1	The transport formula	15
7.2	Lifting a negative result	16
7.3	A controlled dimension experiment	16
8	A concrete Lean integration plan	17
8.1	The object-level interface	17
8.2	Source admission and reification	17
8.3	Reuse specific libraries rather than reimplement their objects	18
8.4	The narrow influence of Leant and Djex	19
8.5	Proposed files and acceptance obligations	19
9	What was actually tested	20
9.1	Execution and units	20
9.2	Families and independent checks	20
9.3	Adversarial controls	21
9.4	What the evidence does and does not establish	22
10	Applications beyond a homology calculator	22
10.1	Exact discrete integration	22
10.2	Finite linear gluing problems	22
10.3	Verification of chain-level algorithms	23
10.4	Mapping cones as a later compiler target	23
11	Implementation priorities and falsifiable milestones	23
11.1	First: a new source theorem, not another contract record	23
11.2	Second: reusable coordinates and quotient transport	23
11.3	Third: reduction and end-to-end transport	24
11.4	Evaluation against real tactics	24
11.5	Scope that should remain outside the first release	24
12	Conclusion	24
A	Receipt schemas and program entry points	25
B	Reproducing the evidence and reading the archive	26
C	Status ledger	26

1 The extension, and the boundary of the audit

1.1 Do not build another version of the existing proposal

The current Forge repository is no longer the initial proposal for a larger `grind`. Its merged status register includes three rounds of contributions. In particular, integer-lattice extraction and divisibility obstructions, observable quotients, several kinds of closure, and exact quantitative workers are already part of the baseline [2, 3]. This article therefore does *not* claim a new integer equation solver, a new certificate-first architecture, or a new policy for interpreting bounded search failure.

The missing package of capabilities in the reviewed register is more specific: *coordinates and witnesses for cycles modulo boundaries, together with homotopies that preserve those constructions under reduction*. A chain-complex worker is useful because its outputs are related objects. A cycle filler, an induced quotient map, a reduction, and a homotopy are not interchangeable certificates, but each can be used to construct the others under precise hypotheses. The central engineering task is to preserve these relationships through one source-bound interface.

Existing prerequisite	Contribution here	Explicitly not claimed
Integer lattice solving	Compatible cycle/boundary coordinates and universal filling recipes	Smith normal form as a new algorithm
Finite observable quotients	Integral quotient maps retaining torsion relations and representatives	Commuting diagrams as a new abstraction
Certified simplification	Explicit chain reductions (f, g, h) and witness transport	New algebraic Morse theory
Typed witness search	Coupled, degree-correct homotopy synthesis and an adjoint obstruction checker	Solving Leant’s open Church-encoding queries
Certificate admission	Source-specific bases, grading, and coefficient-ring obligations	A new general trust policy

Table 1: The proposed delta; the left column is reused, not resubmitted.

The mathematics of Smith reduction and algebraic cancellation is established. Algebraic Morse theory already treats chain complexes over rings [13]. What is proposed for Forge is a tightly scoped, testable compilation target using these methods, not a new branch of homological algebra.

1.2 What was inspected

The audit used the Forge README, the implementation-status register, the Leant/Djex integration review, the opening structural-search development, the complete conclusion, and the recursive repository inventory. The register and conclusion provide the merged account of all 27 submissions. The neighboring projects’ READMEs and current revision metadata were also read [1, 2, 5, 4, 3, 6, 7].

This is a targeted audit of the merged design, not a claim to have read every line of every frozen original submission. Repository searches for the proposed subject did not return matches, but that is not treated as a proof of absence: index coverage can be incomplete. The defensible novelty claim is that the *combined worker specified here is not listed in the reviewed merged capability account*. The source inventory and exact revision identifiers are recorded in `docs/AUDIT.md`.

Forge’s own merge has compiled some core Lean specimens. It would be incorrect to say that no

Forge source has ever compiled. It remains equally important that this article’s Python checker is not a Lean checker and that the present experiments do not close Forge’s end-to-end implementation gap.

1.3 A source fragment, not a competing foundation

The first accepted inputs should be bounded chain complexes of explicitly based, finite free $\mathbb{Z}$ -modules. All matrices are integral and concrete; vectors may remain universally quantified in the source theorem. This already admits statements such as

$$\forall z \in \mathbb{Z}^n, \quad Bz = 0 \implies \exists w \in \mathbb{Z}^p, \quad Aw = z,$$

or a request to construct a chain homotopy between specified maps. The worker is not intended to search arbitrary topological spaces, infer an unknown finite model of an infinite object, or decide chain-map equivalence over an arbitrary ring.

A general Lean module can be admitted only when a finite basis and the coordinate compatibility proofs are available. Those proofs are obligations, not properties inferred from a type’s printed name. Section 8 gives the proposed source compiler and the existing libraries it should use.

2 A running example: a cycle whose double can be filled

Consider the matrices

$$B = \begin{pmatrix} -1 & -1 & 0 \\ 1 & 0 & -1 \\ 0 & 1 & 1 \end{pmatrix}, \quad z = \begin{pmatrix} 1 \\ -1 \\ 1 \end{pmatrix}, \quad A = 2z.$$

The three columns of B are the boundaries of oriented edges 01, 02, 12. The vector z goes once around the triangle. Since $Bz = 0$, the map A followed by B is zero. We have a chain segment

$$\mathbb{Z} \xrightarrow{A} \mathbb{Z}^3 \xrightarrow{B} \mathbb{Z}^3.$$

Here the single generator in the left-hand group has boundary *twice* the cycle. This is an algebraic doubled-face complex, not the boundary matrix of an ordinary single simplicial triangle.

Every cycle is an integer multiple of z : the first two equations of $Bx = 0$ give $x_2 = -x_1$ and $x_3 = x_1$. The boundaries are precisely the even multiples. Consequently,

$$\ker B / \operatorname{im} A \cong \mathbb{Z}/2\mathbb{Z}.$$

A useful proof worker should produce more than this group name. For the query $Aw = z$, it can return the row vector

$$\phi = (0, 0, 1), \quad \phi A = 2 \equiv 0 \pmod{2}, \quad \phi z = 1 \not\equiv 0 \pmod{2}.$$

This is a finite certificate that no integer filler exists. For the query $Aw = 2z$, the worker returns $w = 1$. The two outcomes fit together: the class of z has order exactly two.

There are three useful controls. Replacing A by z makes every cycle a boundary. Removing A leaves a free infinite-order cycle. Replacing $\mathbb{Z}$ by $\mathbb{Q}$ makes the original cycle fillable by $w = 1/2$. Thus a

certificate cannot be rebound to another coefficient ring or to another differential merely because the problem looks similar.

All the claims above have direct scalar proofs. Their purpose is to state what the automated interface must preserve when there are hundreds of generators, unhelpful bases, and maps between several complexes. The delivered `demo.py` runs the doubled-face example and the homotopy obstruction of Section 6.4.

3 Integral coordinate certificates

Status: Implemented in Python; certificate principles proved below. Lean reflection and quotient construction are proposed, not compiled.

3.1 The exact input and the normal-form receipt

Fix

$$A \in \mathbb{Z}^{n \times p}, \quad B \in \mathbb{Z}^{m \times n}, \quad BA = 0.$$

Vectors are columns, and a product BA means first apply A , then B . Every normal-form receipt for a matrix M contains

$$(S, U, U^{-1}, V, V^{-1})$$

with exact integer entries. The checker verifies both inverse identities for each square change of basis, then verifies

$$UMV = S. \tag{1}$$

It also checks that S is rectangular diagonal, its nonzero diagonal entries are positive and precede all zeros, and consecutive nonzero entries divide one another. This makes integral invertibility a multiplication check, not an unverified determinant claim. Both $0 \times q$ and $q \times 0$ matrices retain their dimensions in the serialized representation.

The search calls SymPy's lower-level `sympy.polys.matrices.normalforms.smith_normal_decomp`; its version-1.14.0 source returns the diagonal matrix and both transformation matrices [8]. A diagonal-only API is inadequate for the witness-producing interface. The search computes the inverse matrices and exports them; the checker neither trusts those inverses nor repeats normal-form search.

3.2 First transformation: the entire integral kernel

Take a receipt

$$UBV = D,$$

where r diagonal entries of D are nonzero. Put $k = n - r$ and define

$$K = V_{[:,r:n]} \in \mathbb{Z}^{n \times k}, \quad L = (V^{-1})_{[r:n,:]} \in \mathbb{Z}^{k \times n}. \tag{2}$$

The slice notation uses zero-based half-open indices, as does the implementation.

Lemma 3.1 (Integral kernel coordinates). *The maps $K : \mathbb{Z}^k \rightarrow \mathbb{Z}^n$ and $L : \mathbb{Z}^n \rightarrow \mathbb{Z}^k$ satisfy $BK = 0$, $LK = I_k$, and*

$$Bz = 0 \implies KLz = z.$$

In particular, the columns of K are a basis of the whole integral kernel, not a finite-index sublattice of it.

Proof. Write $t = V^{-1}z$. Since U is invertible, $Bz = 0$ is equivalent to $Dt = 0$. For $i < r$, the i th equation is $D_{ii}t_i = 0$. Its nonzero integer coefficient forces $t_i = 0$. All remaining coordinates are free. Thus every cycle is $K(t_r, \dots, t_{n-1})^T$, with coordinates Lz . The identities $BK = 0$ and $LK = I_k$ follow by multiplying the indicated columns and rows of the checked change-of-basis matrices. $\square$

This is where integrality matters. A rational nullspace basis whose denominators are cleared columnwise need not automatically generate the entire integer kernel. The checked unimodular transformation avoids a separate saturation assumption.

3.3 Second transformation: boundaries inside the kernel

Because $BA = 0$, define the coordinate boundary matrix

$$C = LA \in \mathbb{Z}^{k \times p}.$$

The checker additionally verifies $KC = A$. This identity is logically implied by the kernel argument but is cheap and useful as an implementation guard. Take a second receipt

$$PCQ = S,$$

where the positive diagonal entries are $d_0, \dots, d_{s-1}$ and $d_i \mid d_{i+1}$. Define

$$X = PL : \mathbb{Z}^n \rightarrow \mathbb{Z}^k, \quad R = KP^{-1} : \mathbb{Z}^k \rightarrow \mathbb{Z}^n. \quad (3)$$

These are the cycle-coordinate map and the representative map. The checker verifies

$$XR = I_k, \quad BR = 0, \quad AQ = RS. \quad (4)$$

For every cycle z , Lemma 3.1 also gives $RXz = z$. The stored matrices therefore carry a reusable finite description of all cycles and all their relations.

Theorem 3.2 (Constructive homology classification). *A valid pair of receipts above induces an isomorphism*

$$\ker B / \operatorname{im} A \cong \left(\bigoplus_{i=0}^{s-1} \mathbb{Z}/d_i\mathbb{Z} \right) \oplus \mathbb{Z}^{k-s}.$$

It sends a cycle z to the first s coordinates of Xz modulo d_i , followed by its remaining integer coordinates. The inverse sends the class of a coordinate vector y to the class of Ry .

Proof. First, R produces cycles because $BR = 0$. If $z = Aw$, then $Xz = PLAw = PCw = SQ^{-1}w$. Thus X maps boundaries into the diagonal relation subgroup $S\mathbb{Z}^p$. Conversely, if $Xz = St$ and $Bz = 0$, then

$$z = RXz = RSt = AQt,$$

so z is a boundary. The maps consequently descend to the quotient and are inverse there: $XR = I$ on all coordinate vectors, while $RX = I$ on cycles. The quotient of $\mathbb{Z}^k$ by $S\mathbb{Z}^p$ is the displayed product of cyclic and free groups, coordinate by coordinate. Factors with $d_i = 1$ are trivial. $\square$

Remark 3.3 (Why use two compatible normal forms?). For this free integral setting, the abstract torsion factors can also be recovered from A alone, together with the rank information from B . Indeed, $\ker B$ is a direct summand of $\mathbb{Z}^n$, so a suitable ambient basis makes A a kernel-coordinate matrix with additional zero rows. The two-stage construction is not needed to invent new numerical invariants. It is needed here to keep *specified representatives, inverse coordinates, fillers, and induced maps* compatible in one certificate.

3.4 A complete filler-or-obstruction algorithm

Given a cycle z , compute $y = Xz$. A filler exists precisely when

$$d_i \mid y_i \ (i < s), \quad y_i = 0 \ (s \leq i < k). \quad (5)$$

If these conditions hold, let $t \in \mathbb{Z}^p$ have first s entries y_i/d_i and all other entries zero, and return

$$w = Qt. \quad (6)$$

Replay of an individual filler requires only $Aw = z$, not either normal form. The classifier can therefore do expensive work once and emit very small per-query receipts.

When a condition fails, choose a failing coordinate i and let ϕ be row i of X . For $i < s$, return modulus d_i ; for $i \geq s$, return modulus zero, which the protocol uses to mean exact equality in $\mathbb{Z}$.

Theorem 3.4 (Direct obstruction certificate). *Suppose either $\phi A = 0$ and $\phi z \neq 0$, or for an integer $q \geq 2$,*

$$\phi A \equiv 0 \pmod{q}, \quad \phi z \not\equiv 0 \pmod{q}.$$

Then no $w \in \mathbb{Z}^p$ satisfies $Aw = z$. For a cycle in the admitted finite free integral fragment, the coordinate algorithm returns either a valid filler or such an obstruction, provided its normal-form computation completes without a resource cutoff.

Proof. If $Aw = z$, multiplication by ϕ contradicts the indicated separation, either exactly or modulo q . For completeness, the successful divisibility case was proved in Theorem 3.2. Otherwise a failing row of X annihilates boundaries in the required sense because $XA = SQ^{-1}$. That row separates z by the failed condition. A failed modulus cannot be one, since every integer is divisible by one. $\square$

A modular witness proves nonexistence over $\mathbb{Z}$. It does not license the same negative answer over $\mathbb{Q}$, nor a claim that the cycle itself is nonzero as an ordinary vector modulo every prime.

3.5 Universal exactness, class orders, and useful asymmetries

The same receipt settles more than individual queries.

Corollary 3.5 (A universal filling map). *Every cycle is a boundary if and only if $s = k$ and $d_i = 1$ for every $i < k$. When this holds, let $J : \mathbb{Z}^k \rightarrow \mathbb{Z}^p$ include the first k coordinates. Then $H = QJX$ is an integral linear map such that*

$$Bz = 0 \implies AHZ = z.$$

Proof. The classification is zero exactly under the stated conditions. In that case $SJ = I_k$, so $AH = RSJX = RX$, which acts as the identity on cycles. $\square$

This is the right compiled result for the universally quantified exactness statement in Section 1. It does not enumerate values of z . The prototype exports the coordinates from which this map is obtained; a separate Lean theorem constructor for universal exactness is not implemented.

There is a subtle trap. Exactness at the middle object does *not* in general imply an ambient identity $AH + EB = I_n$ with integral matrices H, E . Take A to have no columns and take $B = (2)$. Its kernel is zero, so middle exactness holds, but $2E = 1$ has no integer solution. An integration that silently replaces the cycle-restricted condition with this stronger identity would reject valid exactness problems or rationalize away an obstruction.

For a cycle with coordinates y , its class has infinite order exactly when a free coordinate is nonzero. Otherwise its order is

$$\text{lcm}_{0 \leq i < s} \frac{d_i}{\gcd(d_i, y_i)},$$

with the empty least common multiple equal to one. This follows because an integer t kills the class exactly when every d_i divides ty_i . This is a derived use of the proved coordinate certificate, not a separately implemented certificate tag in the supplied package.

3.6 Complexity and certificate size

The prototype's two normal-form searches operate on B and C , with shapes $m \times n$ and $(n-r) \times p$. They are exact integer computations, not floating-point rank tests. The search may experience coefficient growth; this article makes no polynomial bit-complexity claim for SymPy's implementation.

For dimensions bounded by N , each receipt has $O(N^2)$ integer entries and naive replay uses $O(N^3)$ integer arithmetic operations, with a constant number of matrix products. These are arithmetic-operation bounds: operand bit lengths also matter. Individual fillers and obstructions are smaller, requiring matrix-vector or row-matrix products. For many queries on the same segment, caching the source-bound coordinate certificate is therefore more useful than repeating Smith reduction.

4 Maps on homology: preserve relations, not just group names

Status: Implemented and tested, including changes of basis and coordinate composition. The Lean quotient-map adapter remains proposed.

Suppose a second segment is $\mathbb{Z}^{p'} \xrightarrow{A'} \mathbb{Z}^{n'} \xrightarrow{B'} \mathbb{Z}^{m'}$, and a proposed map of segments contains

$$F : \mathbb{Z}^n \rightarrow \mathbb{Z}^{n'}, \quad E : \mathbb{Z}^m \rightarrow \mathbb{Z}^{m'}, \quad G : \mathbb{Z}^p \rightarrow \mathbb{Z}^{p'}.$$

The checker requires both squares:

$$B'F = EB, \quad FA = A'G. \tag{7}$$

The first ensures cycles map to cycles; the second ensures boundaries map to boundaries. The distinction matters: checking only one square is insufficient.

Let (X, R, S, Q) and (X', R', S', Q') be the two accepted coordinate systems. The matrix of the induced map in the presented quotient coordinates is

$$M = X'FR. \tag{8}$$

It is accompanied by a relation-transport matrix

$$T = (Q')^{-1}GQ.$$

These matrices satisfy

$$MS = S'T. \tag{9}$$

Indeed, $RS = AQ$, so $MS = X'FAQ = X'A'GQ = S'(Q')^{-1}GQ$. Equation (9) states exactly that changing a source representative by a boundary changes its image by a target boundary.

Proposition 4.1 (Functorial coordinate transport). *The matrices in (8) define the intended homology maps. For composable maps F_1, F_2 and a fixed coordinate system on their common object,*

$$M(F_2F_1) = M(F_2)M(F_1).$$

The identity map induces I in the full coordinate presentation.

Proof. Well-definedness follows from (9); representatives are sent to the correct cycles by construction. The columns of F_1R_0 are cycles in the middle complex, so $R_1X_1F_1R_0 = F_1R_0$. Thus

$$X_2F_2R_1X_1F_1R_0 = X_2F_2F_1R_0.$$

For identities, use $XR = I$. □

The word “full” is important. The implementation retains coordinates whose relation is 1, even though they represent trivial cyclic factors. Removing those coordinates changes matrix dimensions and requires another explicitly checked presentation map. The displayed group summary omits trivial factors; the algebraic transport does not silently omit their coordinates.

A later equality worker can check whether two induced maps agree in the quotient without demanding literal equality of their coordinate matrices. If their difference is N , equality means each column of N lies in $S'\mathbb{Z}P'$: divisibility in torsion rows and exact zero in free rows. This criterion is proved by the same coordinate argument. It is a proposed frontend operation, not an extra experimental outcome counted in this article.

5 Reduction with retained chain homotopies

Status: Greedy unit cancellation, composite reduction receipts, and direct replay are implemented. Matrices in this prototype are dense, not a sparse production representation.

5.1 Why an invariant alone is not a sufficient reduction result

A preprocessing routine that returns a smaller complex with the same homology has not necessarily returned enough information to reconstruct a proof about the original source. For a filler, a homotopy, or an induced map, the controller needs actual maps between the two representations.

Represent a bounded complex by its total finite free module $C = \bigoplus_j C_j$, an ordered integer degree for every basis vector, and a matrix d of degree -1 with $d^2 = 0$. The degree list is part of the source problem. A reduction to $(\bar{C}, \bar{d})$ consists of maps

$$f : C \rightarrow \bar{C}, \quad g : \bar{C} \rightarrow C, \quad h : C \rightarrow C$$

of degrees $0, 0, +1$ satisfying

$$fd = \bar{d}f, \quad dg = g\bar{d}, \quad (10)$$

$$fg = I_{\bar{C}}, \quad gf = I_C - dh - hd. \quad (11)$$

These are the exact equations checked by the prototype. Additional side conditions such as $h^2 = 0$, $fh = 0$, or $hg = 0$ are *not* required by the receipt language. Nothing in the following transport arguments needs them. Thus the article does not silently use a stronger normalized-contraction contract than the checker actually enforces.

Proposition 5.1 (Reduction preserves homology constructively). *A receipt satisfying (10)–(11) induces inverse isomorphisms on homology. If z is a cycle and $\bar{d}\bar{w} = fz$, then*

$$w = g\bar{w} + hz \quad (12)$$

is a filler of z in the original complex.

Proof. Chain-map laws preserve cycles and boundaries. On a cycle, $z - gfgz = dhz$ is a boundary, while $fg = I$ holds literally. This gives inverse maps on homology. For the explicit filler,

$$dw = dg\bar{w} + dhz = g\bar{d}\bar{w} + dhz = gfgz + dhz = z,$$

where the final equality uses $dz = 0$ in (11). □

This small formula is a major practical reason to retain h . A rank comparison alone cannot construct its correction term.

5.2 Single-pair cancellation over the integers

Suppose a differential entry $d_{\tau\sigma} = u$ is a unit, hence $u = 1$ or $u = -1$. Necessarily $\deg \sigma = \deg \tau + 1$. Define h to have one nonzero entry $h_{\sigma\tau} = u^{-1} = u$. Remove the two basis vectors, and let E be the inclusion of the surviving coordinate vectors and $P_0 = E^T$ their projection. Set

$$\bar{d} = P_0(d - dh)dE, \quad (13)$$

$$f = P_0(I - dh), \quad g = (I - hd)E. \quad (14)$$

The formulas use no nonintegral division. Cancelling an entry 2 as though it were a unit would destroy the doubled-face obstruction; the search explicitly refuses such a pivot.

Theorem 5.2 (Unit-cancellation certificate). *The matrices in (13)–(14), together with the single-entry h , satisfy the reduction equations, and $\bar{d}^2 = 0$.*

Proof. Reorder the basis as τ, σ , survivors. By grading, the differential has block form

$$d = \begin{pmatrix} 0 & u & \alpha \\ 0 & 0 & \beta \\ \gamma & \delta & \epsilon \end{pmatrix}.$$

The equations $\bar{d}^2 = 0$ include

$$\alpha\delta = 0, \quad u\beta + \alpha\epsilon = 0, \quad \gamma u + \epsilon\delta = 0, \quad \gamma\alpha + \delta\beta + \epsilon^2 = 0.$$

Direct substitution into the proposed formulas gives

$$\bar{d} = \epsilon - \delta u^{-1} \alpha, \quad f = \begin{pmatrix} -\delta u^{-1} & 0 & I \end{pmatrix}, \quad g = \begin{pmatrix} 0 \\ -u^{-1} \alpha \\ I \end{pmatrix}.$$

Now $fg = I$. Multiplying the displayed blocks gives $gf = I - dh - hd$, using $\alpha\delta = 0$. The second and third square-zero identities above give $fd = \bar{d}f$ and $dg = g\bar{d}$. Finally,

$$\begin{aligned} \bar{d}^2 &= \epsilon^2 - \epsilon\delta u^{-1} \alpha - \delta u^{-1} \alpha \epsilon + \delta u^{-1} \alpha \delta u^{-1} \alpha \\ &= (-\gamma\alpha - \delta\beta) + \gamma\alpha + \delta\beta + 0 = 0. \end{aligned}$$

The inherited degree list verifies all grading conditions. □

This is a local algebraic cancellation, related to the established algebraic Morse framework [13]. The prototype's contribution is not the formula's novelty; it is the complete reduction receipt and its use as an input to other Forge workers.

5.3 Composition without retaining the search history

If (f_1, g_1, h_1) reduces C to C_1 and (f_2, g_2, h_2) reduces C_1 to C_2 , compose them as

$$f = f_2 f_1, \quad g = g_1 g_2, \quad h = h_1 + g_1 h_2 f_1. \quad (15)$$

The chain-map laws and $fg = I$ are immediate. For the homotopy law,

$$g_1 g_2 f_2 f_1 = g_1 (I - d_1 h_2 - h_2 d_1) f_1 = I - d(h_1 + g_1 h_2 f_1) - (h_1 + g_1 h_2 f_1) d.$$

Thus the final checker can verify only the final maps, without trusting the sequence of pivots or retaining an enormous proof trace. Intermediate receipts remain useful for diagnostics, but are not necessary for the final mathematical claim.

A production implementation should compare this dense composite receipt with a factorized sequence of elementary reductions. The former is simple to check; the latter may avoid coefficient and storage growth. A factorized checker would need to verify every local step and compose its semantics, rather than assume that a compressed history expands correctly. Only the dense final form is implemented here.

5.4 Pivot policy, stopping, and a deliberate failure of the heuristic

The producer scans entries of absolute value one and chooses a pivot minimizing

$$(\text{nonzeros in its row} - 1)(\text{nonzeros in its column} - 1),$$

with deterministic index tie-breaking. This is a fill-in heuristic, not a minimality theorem. Each cancellation removes two basis vectors, so absent a step cap there are at most $\lfloor N/2 \rfloor$ cancellations for N total basis vectors. The supplied implementation is dense and recomputes products; its asymptotic efficiency should not be inferred from the heuristic's name.

When the step cap is reached, the producer returns a *valid partial reduction*. It does not claim that the surviving complex is minimal, that its differential has no units, or that any requested homotopy is impossible. A one-step-capped augmented tetrahedron is included in the tests.

A stronger boundary test uses the two-term differential

$$M = \begin{pmatrix} 2 & 3 \\ 3 & 5 \end{pmatrix}.$$

It has no unit entry, so the greedy cancellation worker performs no reduction. Nevertheless $\det M = 1$ and

$$M^{-1} = \begin{pmatrix} 5 & -3 \\ -3 & 2 \end{pmatrix}$$

is integral. The associated two-term complex is contractible. The whole-diagram homotopy worker finds a contraction even though the local pivot heuristic stagnates. Both observations are recorded, rather than scoring the stalled reduction as a mathematical failure.

A natural next optimization is a checked unimodular row/column preconditioner that exposes unit factors before cancellation. It would update adjacent differentials by the corresponding inverse basis changes and emit those changes explicitly. This optimization is not implemented or benchmarked here.

5.5 What reduction does not establish

A chain contraction is an algebraic statement about the supplied differential. It is not, by itself, a certificate of topological contractibility of a space. Acyclicity, a chain-homotopy equivalence, a simple-homotopy equivalence, and a topological homotopy equivalence are different claims. The prototype accepts only the first two kinds of algebraic data through its stated matrix equations. It does not infer a topological theorem from a matching homology summary.

6 Whole-diagram homotopy synthesis and dual obstruction

Status: Both constructive and negative searches are implemented. Negative replay uses an adjoint matrix identity, not the search-side equation assembler.

6.1 The synthesis problem

Let (C, d_C) and (D, d_D) be bounded finite free integral complexes, and let $F, G : C \rightarrow D$ be degree-zero chain maps. The checker first requires

$$d_D F = F d_C, \quad d_D G = G d_C.$$

The task is to construct an integral map H of degree +1 such that

$$F - G = d_D H + H d_C. \tag{16}$$

This is the chain-homotopy equation in column-vector convention. Mathlib already has the mathematical object `Homotopy`, its composition operations, and the theorem that homotopic maps induce equal homology maps [12]. The proposed worker supplies the missing constructive matrix search and its source-bound reconstruction.

A degree-wise greedy approach can make incompatible choices. A component H_j appears both in $d_{D,j+1} H_j$ and in the neighboring equation $H_j d_{C,j+1}$. The whole system must therefore be assembled with shared unknowns. This is exactly the kind of correlated witness selection that a proof planner must preserve rather than split into unrelated local synthesis requests.

6.2 A concrete integer system

Enumerate the allowed entries of H :

$$\mathcal{V} = \{(a, b) : \deg_D(a) = \deg_C(b) + 1\}.$$

Enumerate the degree-zero equation entries

$$\mathcal{E} = \{(i, j) : \deg_D(i) = \deg_C(j)\}.$$

For a variable H_{ab} , its coefficient in equation (i, j) is

$$L_{(i,j),(a,b)} = (d_D)_{ia}[j = b] + [i = a](d_C)_{bj}, \quad (17)$$

where brackets denote indicators. Set $b = \text{vec}_{\mathcal{E}}(F - G)$ and solve $Lx = b$ over $\mathbb{Z}$ using an ordinary Smith/lattice solver. This inner solver is reused infrastructure; the contribution is the whole-diagram compiler and the distinct reconstruction/obstruction contracts.

The number of variables is

$$v = \sum_j \text{rank } C_j \text{ rank } D_{j+1},$$

and the number of scalar equations is

$$e = \sum_j \text{rank } C_j \text{ rank } D_j.$$

The prototype checks its equation and variable caps *before* constructing a large system. An empty equation set and an empty variable set are distinct shapes, both of which occur in the tests.

A successful integer solution is reshaped into H and returned. Its final checker does not inspect the Smith certificate or the assembled system: it verifies the grading and the original matrix equation (16) directly.

6.3 Negative answers that can be checked without the assembler

Suppose the lattice solver reports a separating row functional λ for $Lx = b$, either exactly or modulo $q \geq 2$. Rather than serialize L and ask a second routine to build it in the same way, reshape λ into a degree-zero matrix W of the same shape as F and G . Define the integer Frobenius pairing

$$\langle W, T \rangle = \sum_{i,j} W_{ij} T_{ij}.$$

The checker evaluates the *adjoint* matrix

$$d_D^T W + W d_C^T. \quad (18)$$

Theorem 6.1 (Adjoint homotopy obstruction). *Assume W has degree zero. If*

$$d_D^T W + W d_C^T = 0, \quad \langle W, F - G \rangle \neq 0,$$

then no integral H satisfies (16). The same conclusion holds when the first identity and the nonzero test are interpreted modulo a fixed integer $q \geq 2$.

Proof. For every matrix H of the required shape,

$$\begin{aligned}\langle W, d_D H + H d_C \rangle &= \langle d_D^T W, H \rangle + \langle W d_C^T, H \rangle \\ &= \langle d_D^T W + W d_C^T, H \rangle.\end{aligned}$$

The first equality follows by expanding finite sums and exchanging their indices. Under either the exact or the modular hypothesis the final pairing vanishes, whereas $\langle W, F - G \rangle$ does not. This contradicts (16). The grading ensures that the adjoint occupies exactly the allowed degree-+1 positions; requiring its entire matrix to vanish therefore does not lose a legitimate dual witness. $\square$

This is a deliberately different computation from the search. The producer loops over equation/-variable index pairs and performs Smith reduction. The checker multiplies the original differential matrices and evaluates a pairing. A mistaken index convention in the assembler need not be replicated by the checker. The two components still share the mathematical specification and JSON schema, so this engineering separation is not a formal proof of either implementation's correctness.

Theorem 6.2 (Completeness of the uncapped finite integral fragment). *For fixed finite free integral complexes and fixed chain maps, the homotopy procedure without resource caps returns either a valid H or a valid exact or modular adjoint obstruction W .*

Proof. The allowed entries of H form $\mathbb{Z}^v$, and equation (17) represents (16) exactly. Let $ULV = S$ be a Smith decomposition and $y = Ub$. If all nonzero diagonal entries of S divide their corresponding entries of y , and all trailing entries of y vanish, the usual transformed solution gives x and hence H . Otherwise a row of U gives a functional annihilating L exactly or modulo a failed diagonal divisor, but not annihilating b . Reshaping that row gives W . The coefficient of each allowed entry of H in its pairing with the right side of (16) is the corresponding entry of (18). Thus the dual conditions hold. Smith reduction of a finite integer matrix terminates; no enumeration of unbounded candidate homotopies is needed. $\square$

The implemented caps weaken this to a three-way computational outcome: a witness, an obstruction, or UNKNOWN. An exception, exhausted memory, unsupported coefficient type, or cap refusal is not a fourth kind of proof.

6.4 Equal homology maps are not enough

The following example is both a mathematical separator and an acceptance test for the intended integration. Let C be concentrated in degrees 1, 0, with $d_C : C_1 \rightarrow C_0$ multiplication by $p \geq 2$. Let D be concentrated in degrees 2, 1, with $d_D : D_2 \rightarrow D_1$ multiplication by p . Define $F : C \rightarrow D$ by $F_1 = t$ and all other components zero; let $G = 0$.

The only nonzero homology of C is $H_0(C) = \mathbb{Z}/p\mathbb{Z}$. The only nonzero homology of D is $H_1(D) = \mathbb{Z}/p\mathbb{Z}$. Therefore F and G induce the same, zero maps on *every* homology group, for every t .

A chain homotopy has two potentially nonzero components:

$$H_0 : C_0 \rightarrow D_1, \quad H_1 : C_1 \rightarrow D_2.$$

Writing them as integers a, b , equation (16) reduces to

$$pa + pb = t. \tag{19}$$

Hence F is null-homotopic if and only if $p \mid t$. For $p = 2, t = 1$, the desired homotopy does not exist even though every induced homology map agrees with zero.

In total-module coordinates, with source degrees $(0, 1)$ and target degrees $(1, 2)$, the certificate is particularly small:

$$d_C = d_D = \begin{pmatrix} 0 & p \\ 0 & 0 \end{pmatrix}, \quad F = \begin{pmatrix} 0 & t \\ 0 & 0 \end{pmatrix}, \quad W = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}.$$

Then $d_D^T W + W d_C^T = pI_2$ and $\langle W, F \rangle = t$. Modulo p , these are precisely the conditions of Theorem 6.1.

This example rules out two tempting but invalid shortcuts: treating equality of homology maps as a sufficient criterion for homotopy, and using a rational homotopy to answer an integral existence question. The delivered tests cover $p = 2, \dots, 9$ and $t = -2p, \dots, 2p$, retaining both positive and negative outcomes. The routine is not merely being tested on cases where a homotopy was constructed in advance.

7 Reduction-first synthesis with certified lifting

7.1 The transport formula

Suppose source and target reductions have been checked:

$$(f_C, g_C, h_C) : C \rightleftarrows \bar{C}, \quad (f_D, g_D, h_D) : D \rightleftarrows \bar{D}.$$

Set $T = F - G$ and

$$\bar{T} = f_D T g_C.$$

Rather than solve on the original complexes, ask for $\bar{H}$ such that

$$\bar{T} = \bar{d}_D \bar{H} + \bar{H} \bar{d}_C.$$

The original homotopy is reconstructed by the explicit formula

$$H = h_D T + g_D \bar{H} f_C + g_D f_D T h_C. \tag{20}$$

All three terms matter. Dropping the two correction terms generally proves only something about a reduced surrogate of the requested map.

Theorem 7.1 (Homotopy lifting). *Under the checked reduction equations, the matrix in (20) satisfies $d_D H + H d_C = T$.*

Proof. Since T is a chain map,

$$T - g_D f_D T = (d_D h_D + h_D d_D) T = d_D (h_D T) + (h_D T) d_C.$$

Because $g_D f_D T$ is also a chain map,

$$g_D f_D T - g_D f_D T g_C f_C = d_D (g_D f_D T h_C) + (g_D f_D T h_C) d_C.$$

Finally, applying the chain-map laws to the reduced homotopy gives

$$g_D f_D T g_C f_C = g_D \bar{T} f_C = d_D (g_D \bar{H} f_C) + (g_D \bar{H} f_C) d_C.$$

Adding the three identities proves the result. Grading is preserved in every term. $\square$

There is also an exact two-way criterion: T is null-homotopic if and only if $\bar{T}$ is. The forward direction sends H to $f_D H g_C$; the reverse direction is Theorem 7.1. Thus reduction changes the computational problem without weakening it, provided the receipts and transport are retained.

7.2 Lifting a negative result

The implemented transport helper lifts positive homotopies. A negative reduced obstruction can also be transported on paper. If $\bar{W}$ separates $\bar{T}$, set

$$W = f_D^T \bar{W} g_C^T. \quad (21)$$

For any candidate H , the pairing with its homotopy boundary equals the pairing of $\bar{W}$ with the reduced homotopy boundary of $f_D H g_C$, by the chain-map identities. Moreover $\langle W, T \rangle = \langle \bar{W}, f_D T g_C \rangle$. Hence the exact or modular separator transports too.

This formula is a specified extension, not part of the recorded transport helper’s executed test claim. A first implementation should replay the lifted W against the original matrices using the same adjoint checker, rather than rely on a special exception for transported certificates.

7.3 A controlled dimension experiment

Take $C = D$ to be a direct sum of q two-term complexes whose differentials are units. Use nontrivial degree-preserving changes of basis, and generate a null-homotopic map from a chosen degree-+1 matrix. The original homotopy system has $2q^2$ scalar equations and q^2 variables. Unit cancellation reduces both complexes to zero; the reduced problem consequently has no equations or variables. Formula (20) still produces a concrete original homotopy, which is checked directly.

q	Cells per complex	Original equations	Variables	Direct search	Lifted result
1	2	2	1	homotopy	checked
2	4	8	4	homotopy	checked
4	8	32	16	homotopy	checked
6	12	72	36	homotopy	checked
8	16	128	64	homotopy	checked
10	20	200	100	homotopy	checked
12	24	288	144	cap: unknown	checked

Table 2: All reduced systems have zero equations and variables. The direct search cap is 256 equations. These are controlled Python cases, not Lean tactic benchmarks.

This result is a capability observation under a stated cap, not a general speedup. The family was chosen to contain cancellable summands; it does not measure how often such summands occur in user developments. The original, reduced, and transported computations have different costs, and a fair latency comparison would include reduction and replay, not just the small solve. Raw direct-search timings are retained in `ablations.json` for reproducibility but are not advertised as a tactic performance result.

A further fixture leaves a nonzero torsion differential after cancellation and uses a nonzero reduced homotopy. It verifies that the middle term $g_D \bar{H} f_C$ is exercised, rather than testing only the degenerate case in which it vanishes.

8 A concrete Lean integration plan

Status: Design only. The named Mathlib APIs were inspected in official reference documentation; no compatibility claim is based on a compiler run.

8.1 The object-level interface

The first implementation should separate a small integer-matrix language from the source-level mathematical objects. A proposed internal record has an ordered finite basis, its degree vector, and a concrete differential. Its validity predicate asserts the degree- -1 condition and $d^2 = 0$. A matrix homotopy record contains H , a degree- $+1$ proof, and the original equation $F - G = d_D H + H d_C$.

The core soundness interfaces should have the following mathematical shape; these are proposed signatures, not declarations already present in Forge:

```
checkCoordinates(problem, receipt) = true
  -> IntegralCoordinates(problem, receipt)

checkReduction(problem, receipt) = true
  -> ReductionWitness(problem, receipt)

checkHomotopy(problem, receipt) = true
  -> Nonempty (IntegralHomotopy problem.F problem.G)

checkNoHomotopy(problem, receipt) = true
  -> Not (Nonempty (IntegralHomotopy problem.F problem.G))
```

A Lean Homotopy $F\ G$ is data, not itself a proposition to which one should blindly apply logical negation. Its non-inhabitation is expressed using `Not (Nonempty ...)`. A positive data-synthesis goal can instead request that structure directly and use the returned matrix entries as its fields.

For homology, `IntegralCoordinates` should first expose the cycle and relation identities of Section 3. The categorical quotient isomorphism can then be constructed by a separate adapter. This keeps the small arithmetic checker from depending on the entire homology library and makes it useful even for users who stated their problem as a matrix exactness condition rather than as a categorical homology object.

8.2 Source admission and reification

The admission procedure should be an explicit sequence:

1. Recognize an exactness, cycle-filling, induced-map, or homotopy goal. Retain the original expressions, coefficient ring, local context, maps, and selected bases.
2. Obtain a finite ordered coordinate equivalence for every source module. For a module already presented as `Fin n -> Int`, this is direct. For an abstract module, require a supplied finite basis and its actual representation map; do not infer one from a dimension annotation.
3. Translate linear maps into matrices using those bases. Normalize entries only when Lean can prove the resulting equalities. The numeric solver accepts concrete integers, not arbitrary symbolic coefficients evaluated at a guessed model.

4. Prove the source-to-matrix compatibility squares, grading, and chain-map conditions. Unresolved obligations remain visible to Forge’s existing planner.
5. Request a certificate, run the reflected matrix checker, apply its soundness theorem, and transport the result through the checked coordinate maps to the original source goal.

The planner may try an abstract library theorem before entering the finite fragment. Failure to obtain a finite basis means the worker is inapplicable, not that the original theorem is false.

The crucial bridge for a source linear map $a : M \rightarrow N$ and coordinate isomorphisms e_M, e_N is

$$e_N(a(x)) = A e_M(x).$$

A source proof can use a computed filler only after this bridge is established. Similarly, transporting a matrix homotopy requires the bridge for both differentials and both chain maps, not just a proof that the matrices have the right dimensions.

Universally quantified vectors do not have to be enumerated. Once A, B and an exactness receipt are fixed, Corollary 3.5 supplies the function $z \mapsto Hz$ for every cycle. A checked contraction can likewise be used by universal transport lemmas on symbolic chain maps. These uses are mathematical consequences of the receipts; their automatic source-level construction remains part of the proposed adapter.

8.3 Reuse specific libraries rather than reimplement their objects

Library or module	Concrete role and scope
SymPy 1.14.0, <code>polys.matrices.normalforms</code>	Executed search backend: <code>smith_normal_decomp</code> produces S, U, V over $\mathbb{Z}\mathbb{Z}$. Its output and integer inverses are checked independently. No floating rank computation is used.
Python standard library	Executed replayer: exact Python integers, a separate matrix implementation, strict JSON parsing, and schema validation. No SymPy import is needed for replay.
SageMath integer matrices	An untested alternative search adapter can use <code>smith_form()</code> and export D, U, V and inverse matrices to the same schema. It should not bypass the checker.
<code>Mathlib.LinearAlgebra. Matrix.ToLin</code>	Existing coordinate infrastructure, including matrix/linear-map conversions such as <code>LinearMap.toMatrix</code> and <code>Matrix.toLin</code> . Use it for basis-aware transport, not ad hoc textual rewriting.
<code>Mathlib.Algebra.Homology. Homotopy</code>	Existing homotopy structures and composition operations. Reconstruct their <code>hom</code> , <code>zero</code> , and <code>comm</code> fields from checked matrix data; then use <code>Homotopy.homologyMap_eq</code> or <code>HomotopyEquiv.toHomologyIso</code> for downstream consequences.

The library descriptions are based on the inspected official references [8, 9, 10, 11, 12]. The Sage path and the Lean path are not executed components of this delivery. The matrix convention is $F - G = dH + Hd$; Mathlib’s documented homotopy `comm` field is arranged as $F = dH + Hd + G$. The adapter must prove that rearrangement and respect categorical composition order rather than paste a matrix product into a field with a different convention.

For the first reflected checker, dense matrices indexed by `Fin` are adequate. Prove multiplication evaluation and the finite-sum pairing identity, then use closed exact arithmetic to check the matrices. A later sparse format should canonicalize duplicate entries and explicitly prove that omitted entries mean zero. The current prototype has no sparse serializer, and no performance claim in this report assumes one.

8.4 The narrow influence of Leant and Djex

Leant and Djex already emphasize source-owned candidates, exact type and provider selections, and replay before admission [6, 7]. Forge’s own neighbor review already incorporates these lessons [5]; repeating them as a new architecture would be redundant. The extension-specific consequence is that a candidate H is owned by the *exact* source/target complexes, selected bases, grading, and coefficient ring used to derive it.

A candidate ranker may prefer smaller matrices, smaller coefficient heights, or shorter source terms. It must not exchange a receipt between two presentations merely because their homology summaries agree. A basis transport is accepted only through explicit compatibility maps. Changing the domain from $\mathbb{Z}$ to $\mathbb{Q}$ is not a cosmetic type adjustment: it can reverse a negative result.

The numeric worker does not run Djex and does not claim to extend its higher-rank search. In a combined system, type-directed synthesis can assemble the existing coordinate and homotopy constructors after the arithmetic worker has supplied their coefficient data. That is a division of labor, not a claim that the original Church-encoding or universe-selection acceptance tests have passed.

8.5 Proposed files and acceptance obligations

A focused Lean implementation can be organized as follows:

```
Forge/IntegralComplex/MatrixData.lean
Forge/IntegralComplex/MatrixCheck.lean
Forge/IntegralComplex/Coordinates.lean
Forge/IntegralComplex/Reduction.lean
Forge/IntegralComplex/HomotopyDual.lean
Forge/IntegralComplex/Reify.lean
Forge/IntegralComplex/MathlibAdapter.lean
```

These are *proposed paths*, not files claimed to exist in the supplied archive. Their responsibilities are deliberately separable. The first two establish executable matrix semantics. The next three prove the certificate principles. Reification binds certificates to source objects. The final adapter uses the existing homology library rather than replacing it.

An initial acceptance test should ask Lean to prove, at the original source type, the nonexistence of a homotopy in the $p = 2, t = 1$ example. The test must also accept a homotopy for $t = 2$, and must not turn the negative answer into a statement over $\mathbb{Q}$. This tests a genuinely distinguishing capability with small proof objects. It is more informative than compiling a record definition whose soundness fields have merely been assumed.

9 What was actually tested

9.1 Execution and units

The recorded run used Python 3.13.5, SymPy 1.14.0, and Linux. The generator seed is 20260915. This is a new generator with its own inputs; sharing a seed value with any previous Forge run does not make the case sets comparable. The case manifest includes family and outcome labels so that a successful negative certificate is not mistaken for a failed proof attempt.

Stored certificate outcome	Cases
Integral homology coordinates	97
Explicit cycle filler	129
Cycle obstruction	65
Induced quotient map	48
Reduction witness	72
Chain homotopy	100
No-homotopy obstruction	145
Total stored certificate cases	656

Table 4: One internally consistent corpus, not a pooling of earlier Forge runs. Each row is a checked certificate outcome.

Every one of these stored objects also replays under `python -S replay.py`. The `-S` flag disables site-package setup; the replayer additionally checks that SymPy is absent from the imported-module table. The result is recorded in `results/replay.log`. The generated-case suite and the dependency-free unit suite are separate commands and have separate logs.

The per-case CSV times the proposal-call wrapper and the checker separately. For precomputed fixtures and transported witnesses, prerequisites are prepared outside that wrapper. Its column is therefore named `proposal_call_seconds`, not end-to-end search time; it is diagnostic instrumentation rather than a performance comparison.

The largest source or receipt matrix dimension in the stored corpus is 24; the largest coefficient magnitude uses 13 bits. The largest compact JSON certificate is 2,024 bytes. These are small exact tests, not evidence about industrial-sized complexes or the input caps. The entire indented corpus occupies 1,586,739 bytes, including source problems and test metadata.

9.2 Families and independent checks

The 96 generated chain segments start from explicitly controlled kernel and boundary coordinates, then apply independent unimodular changes of basis in the three modules. Shapes range from zero to five in each segment dimension. Their prescribed diagonal relations provide an expected homology classification that does not depend on the producer’s chosen Smith basis. The suite includes an explicit boundary for each segment and a separately sampled cycle whose expected status is determined in the original prescribed coordinates.

Normal-form arithmetic is also checked on 64 small matrices through *determinantal divisors*: compute the greatest common divisor of every $k \times k$ minor, and divide consecutive nonzero gcds to obtain the invariant factors. The determinant oracle uses a direct signed-permutation sum, not SymPy,

elimination, or the checker’s matrix arithmetic. The matrices comprise 32 scrambled B matrices and 32 prescribed kernel-coordinate boundary matrices after a domain basis change. This is a small-dimensional differential test, not a scalable normal-form implementation.

The induced-map cases include scalar maps and changes of basis. Twenty-four additional coordinate-composition checks verify that a change and its inverse compose to the actual identity matrix in the retained presentation, not merely to matching ranks or group names. These composition checks reuse the coordinate machinery and should not be described as an independent arithmetic oracle.

The homotopy corpus includes 184 members of the $pa + pb = t$ family, 48 maps constructed from independently chosen degree-+1 matrices, a free obstruction, and empty-complex cases. For each of the 184 tiny family members, an additional oracle enumerates the two homotopy variables modulo p . In this particular family, modular existence agrees with integral existence by (19). That equivalence is *not* asserted for arbitrary integer systems that happen to have a solution modulo one modulus.

The reduction tests use small simplicial boundary matrices, with and without augmentation, and with degree-preserving basis changes; they also use sums of unit, zero, and torsion two-term differentials. The tests retain the valid partial reduction, the nonunit-stagnation fixture, both prerequisite reductions for each transport experiment, and the lifted original homotopies.

Additional test category	Count	Unit
Deliberately invalid inputs/certificates rejected	489	rejection controls
All-minors normal-form comparisons	64	matrix comparisons
Coordinate inverse/composition checks	24	identities
Tiny homotopy-family modular oracles	184	enumerations
Dependency-free unit tests	12	test methods
Reduction-first dimension experiments	7	experiment rows

Table 5: These counts must not be added to the certificate total to create a larger headline success count.

9.3 Adversarial controls

The invalid controls are deliberately constructed to invalidate a required property, not arbitrary perturbations that might still describe a valid certificate. They include wrong claimed free ranks; corrupted inverse matrices; zeroed separating rows; zeroed homotopies for nonzero targets; omitted reduction homotopy corrections; wrong induced-map entries; and violations of either source square.

Boundary controls reject a noncycle, modulus one, a wrong coefficient domain, boolean and floating-point coefficients, duplicate JSON keys, non-finite JSON numbers, malformed shapes, and unexpected fields. A receipt that separates the doubled-face cycle is also rebound to the zero cycle and correctly rejected. This is a concrete source-rebinding test, not a general proof that every source translation is safe.

The JSON parser and checker are research code. They reject many malformed inputs and impose dimension and input bit-length limits, but have not undergone a security audit. Multiplication can increase intermediate bit lengths, and input caps alone are not a CPU or memory sandbox. The production worker should have an external execution budget and a reflected checker with an explicit cost discipline.

9.4 What the evidence does and does not establish

The tests establish that this implementation generated the recorded objects, that the separate checker accepted them, that the listed invalid controls were rejected, and that the independent checks agreed on their small admitted fragments. Search and replay use different arithmetic implementations; the negative homotopy checker also uses a different algebraic computation.

They do not establish the correctness of Python, the checker, SymPy, or the source compiler by formal proof. They do not establish a solved-goal gain over **grind**, Aesop, or Forge’s earlier workers. The cases are synthetic and purpose-built rather than a representative Lean benchmark corpus. No Lean checker, tactic adapter, or generated source theorem was compiled here.

The theorem proofs in this article justify the *mathematical certificate language*. Implementing those proofs in Lean and binding them to an admitted source problem are separate tasks. This distinction is the acceptance boundary, not a reason to discard useful prototype evidence.

10 Applications beyond a homology calculator

10.1 Exact discrete integration

Let A be the vertex-to-edge difference map of the triangle graph:

$$A = \begin{pmatrix} -1 & 1 & 0 \\ -1 & 0 & 1 \\ 0 & -1 & 1 \end{pmatrix}.$$

For edge data u , finding a potential v means solving $Av = u$. The row $\phi = (1, -1, 1)$ satisfies $\phi A = 0$, so ϕu is a period obstruction. For $u = (0, 0, 1)^T$, no potential exists. For data satisfying $u_1 - u_2 + u_3 = 0$, one can choose $v = (0, u_1, u_2)^T$.

This is the same cycle/boundary interface, now read as discrete integration. With faces present, a second matrix expresses local compatibility, and the quotient distinguishes locally compatible data from globally integrable data. The source compiler must prove that its supplied difference and compatibility operators have the stated matrices. It must not infer global integrability merely from local consistency.

The example is a direct mathematical application of the certificate principles; a graph-to-Lean source compiler is not part of the prototype. The test suite’s simplicial boundary generator operates on Python fixtures, not user Lean graph definitions.

10.2 Finite linear gluing problems

Suppose local values, overlap differences, and compatibility constraints are represented by finite free integral modules and specified linear maps A, B with $BA = 0$. A compatible overlap datum is $z \in \ker B$. A global correction exists exactly when $z \in \operatorname{im} A$. The same worker returns a correction or an exact/modular obstruction, together with reusable coordinates for subsequent data on the same diagram.

This suggests an adapter for finite linear gluing problems and explicit cochain presentations. It is not a claim to solve arbitrary sheaf cohomology: the finiteness, freeness, matrices, and representation

laws are input requirements. Nonfree modules and unbounded resolutions require a different admission theorem and possibly a different certificate language.

10.3 Verification of chain-level algorithms

When two algorithms produce chain maps, equality of their output homology summaries is often weaker than the desired chain-level relationship. The homotopy worker can synthesize the actual comparison data. The integral obstruction example shows that it can also refute an overstrong comparison claim rather than silently substitute an easier one.

A certified reduction can act as a reusable preprocessing result for many maps. For example, once (f, g, h) is checked, a new null-homotopy problem can be reduced without recomputing its source complex's entire invariant presentation. The controller should retain that exact reduction receipt and use the transport formula, not retain only a statement that the homology groups are isomorphic.

10.4 Mapping cones as a later compiler target

For a chain map $F : C \rightarrow D$, a mapping-cone differential can be assembled in blocks with a shifted copy of C and an explicit sign on its differential. This makes questions about quasi-isomorphisms and comparison maps accessible to exactness workers. However, the signs, shifts, boundedness assumptions, and bridge from cone exactness to the requested source theorem must all be proved. The present package contains neither a cone compiler nor a quasi-isomorphism acceptance result. It is a concrete next use of the same infrastructure, not an additional delivered lane.

11 Implementation priorities and falsifiable milestones

11.1 First: a new source theorem, not another contract record

The highest-value first step is the reflected adjoint obstruction checker and its pairing theorem. It has a small arithmetic core and a distinguishing source theorem: the $p = 2, t = 1$ pair is not homotopic over $\mathbb{Z}$, despite equality of every induced homology map. A passing milestone requires the exact original Lean source statement, a fully checked proof term, and a recorded axiom inventory. Compiling a structure that assumes its own correctness does not close this milestone.

Pair the negative case with the positive $t = 2$ homotopy. Add domain-switch and basis-switch rejection tests at the source boundary, not just inside JSON. Only after this narrow path is accepted should the frontend advertise support for general admitted integral homotopy requests.

11.2 Second: reusable coordinates and quotient transport

Formalize Lemma 3.1 and Theorem 3.2, then construct the universally quantified filler of Corollary 3.5. The acceptance set should include free homology, torsion, trivial homology, empty dimensions, and a nonprimitive image of B . The $B = (2)$ example must establish the cycle-restricted theorem without demanding the stronger ambient splitting identity.

Next construct induced maps and prove relation descent. A source-level change-of-basis test must retain the selected basis data and commute with composition. Matching the reported invariant factors is not a sufficient test of this layer.

11.3 Third: reduction and end-to-end transport

Formalize unit cancellation and composition of reductions. Then make the controller call the homotopy worker on the reduced diagrams and reconstruct the original proof using Theorem 7.1. The acceptance corpus must include a nonzero reduced homotopy, not only contractible complexes that reduce to zero.

Only this stage earns a claim about the reduction-first planner. A command that prints a smaller matrix without producing the original source proof has not implemented the transport layer. The nonunit-stagnation fixture should remain in the suite as an explicit test that heuristic stalling triggers another worker or UNKNOWN, never a negative conclusion.

11.4 Evaluation against real tactics

After the Lean paths exist, build a small published corpus of actual goals in this source fragment and evaluate fixed versions of **forge**, **grind**, and relevant specialized tactics under the same premise set. Record whole-goal success, elaboration time, checking time, proof size, and axiom policy. Include both original and basis-scrambled statements, and publish the source statements so that near-miss substitutions can be detected.

The meaningful ablations are: no reduction versus checked reduction; homology summary comparison versus whole-diagram homotopy synthesis; rationalized search versus integral certificates; and coordinate receipts versus repeated individual lattice queries. Some ablations are intentionally weaker methods, not fair competitors. Label them as mechanism tests rather than attributing their limits to **grind**. No such Lean evaluation is reported in this article.

11.5 Scope that should remain outside the first release

The finite free integral fragment has a clean source theorem and a complete uncapped algebraic decision path for its homotopy problem. Extending it at once to symbolic coefficient rings, nonfree modules, unbounded complexes, topological realizations, or arbitrary dependent structures would obscure that advantage. Such extensions need new representation theorems and may have very different algorithmic limits.

Within the chosen fragment, practical improvements are already substantial: sparse exact multiplication, factorized transformation certificates, preconditioning that exposes unit factors, dual transport, and caching of proved coordinate systems. Each is independently testable without weakening the acceptance rule.

12 Conclusion

Forge can gain a coherent constructive-exactness worker rather than another unrelated solver. Its distinctive output is not just a verdict or a list of invariants: it is a network of explicit coordinates, representatives, relations, reductions, homotopies, and obstructions that can be transported back to the source theorem.

The integral setting gives the proposal a sharp test. Rational linear algebra loses torsion, equality on homology need not imply a chain homotopy, and greedy unit cancellation need not find every contraction. The supplied algorithms handle these distinctions explicitly. Their receipts are small

enough to check by matrix identities, while their searches may use heavier normal-form machinery. The prototype demonstrates that separation on 656 recorded objects and retains failures of stronger assumptions as regression controls.

The next contribution to the repository should be a reflected Lean checker and one exact source acceptance result, not another expansion of the same design prose. The mathematics, executable reference implementation, and test fixtures here make that step concrete. They do not claim it has already been taken.

A Receipt schemas and program entry points

All serialized matrices use exact integer entries and explicit rectangular shape:

```
{"rows": 2, "cols": 3, "data": [1, 0, -2, 0, 3, 1]}
```

Storage is flat row-major. Booleans are not accepted as integers. Unknown keys are rejected. A certificate does not carry authority to choose its source problem: `check(problem, certificate)` receives the two separately. The stored corpus bundles both for reproducibility, which is not the same as proving correspondence to a Lean source term.

Kind	Certificate payload
<code>homology</code>	Two normal-form receipts, expected free rank, and nontrivial torsion factors. Coordinates X, R are derived and checked.
<code>fill</code>	An integer column w ; check $Aw = z$.
<code>obstruction</code>	An integer row ϕ and modulus zero or at least two; check annihilation of A and separation of z .
<code>reduction</code>	Reduced degree list and differential, plus f, g, h ; check square-zero, grading, chain-map laws, and retraction/homotopy laws.
<code>homotopy</code>	A degree-+1 matrix H ; check the original equation.
<code>no_homotopy</code>	A degree-zero matrix W and modulus; check the adjoint and pairing against the original maps.
<code>induced</code>	Source and target homology receipts and M ; check both source squares, coordinate transport, and quotient relation descent.

The Python producer exposes the following entry points:

```
homology(A, B)
cycle(A, B, z)
reduction(degrees, d)
homotopy(source_degrees, dC, target_degrees, dD, F, G)
induced(A, B, At, Bt, F)
transport_homotopy(Cred, Dred, T, Hbar)
```

The `induced` producer proposes coordinates; the caller's E, G square witnesses are checked in the separately supplied source problem. The transport helper assumes prerequisite reductions have already been checked; its output is then checked on the original homotopy problem.

The search dimension cap is 256. Homotopy assembly separately caps equations and variables at 256 by default; the equation cap is fixed in this prototype. The checker also caps matrix dimensions at 256 and input coefficients at 16,384 bits. These are implementation limits, not mathematical bounds or scalability results. A `SearchLimit` exception means unknown.

B Reproducing the evidence and reading the archive

The archive root is `Forge_Constructive_Exactness`. It contains the article source and PDF, the producer and checker, the complete recorded corpus, per-case timing and size data, rejection and oracle results, the replay log, small worked receipts, and audit/integration notes.

Search and complete regeneration:

```
cd Forge_Constructive_Exactness/prototype
python -m pip install -r requirements.txt
python run_suite.py --output ../reproduced-results
python -S replay.py ../reproduced-results/corpus.json
```

The default regeneration directory differs from `results/recorded`, so ordinary reruns do not overwrite the supplied evidence. Exact timing values will vary. Deterministic certificate content and case counts should match with the pinned search dependency; different normal-form implementations may choose different valid bases.

Replay without search dependencies and small unit tests:

```
python -S replay.py ../results/recorded/corpus.json
python -S -m unittest discover -s tests -v
python demo.py
```

The final command uses SymPy for the demonstration search; the first two do not.

Build the article with ordinary pdfLaTeX:

```
cd ../article
pdflatex -interaction=nonstopmode -halt-on-error \
  forge-constructive-exactness.tex
pdflatex -interaction=nonstopmode -halt-on-error \
  forge-constructive-exactness.tex
```

A third pass may be used after changing the table of contents. No system font files, bibliography processor, or shell escape are required.

C Status ledger

Item	Delivered status
Integral homology coordinates and scalar filler/obstruction search	Python executed
Induced maps and relation descent	Python executed

Item	Delivered status
Greedy unit cancellation and composed f, g, h receipts	Python executed
Whole-diagram integral homotopy and adjoint obstruction search	Python executed
Positive homotopy lifting, including nonzero reduced homotopy	Python executed
Search-free standard-library replay	Python executed
Certificate-language soundness and completeness arguments	Proofs in this article
Class-order and universal-filling corollaries	Proved; no separate receipt tag
Negative homotopy transport	Formula specified; not tested here
Sparse/factorized receipts and unimodular pivot preconditioning	Proposed only
Sage alternative search backend	API inspected; not run
Lean reflected checker and mathematical adapters	Proposed only
Lean source reification, original-goal replay, axiom inventory	Not run
Real Lean tactic comparison	Not run

References

- [1] V. Reshetnikov, *Forge repository README*, revision 58ea206bd7ad501b930add568151217bcc7f82a2, inspected September 15, 2026. Pinned source.
- [2] V. Reshetnikov, *Forge implementation status*, same revision, docs/STATUS.md. Pinned capability register.
- [3] V. Reshetnikov, *Forge merged article: conclusion*, same revision, article/sections/15-conclusion.tex. Pinned conclusion.
- [4] V. Reshetnikov, *Forge: Structural search*, same revision, article/sections/05-structural.tex; opening source section inspected. Pinned source.
- [5] V. Reshetnikov, *Ideas from Leant and Djex*, same revision, docs/IDEAS-FROM-LEANT-DJEX.md. Pinned review.
- [6] V. Reshetnikov, *Leant README*, revision 6bf05ad78c467989e68290f2d08bbbed40802d485, inspected September 15, 2026. Pinned source.
- [7] V. Reshetnikov, *Djex README*, revision e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5, inspected September 15, 2026. Pinned source.
- [8] SymPy developers, *Normal forms of polynomial-domain matrices*, SymPy 1.14.0, sympy/poly/s/matrices/normalforms.py, especially smith_normal_decomp. Version-pinned implementation.
- [9] SymPy developers, *Introducing the DomainMatrix of the polys module*, official SymPy 1.14.0 documentation, inspected September 15, 2026. <https://docs.sympy.org/latest/modules/polys/domainmatrix.html>.

- [10] SageMath developers, *Dense matrices over the integer ring*, official reference manual, `smith_form` documentation, inspected September 15, 2026. https://doc.sagemath.org/html/en/reference/matrices/sage/matrix/matrix_integer_dense.html.
- [11] Mathlib contributors, *Mathlib.LinearAlgebra.Matrix.ToLin*, official API documentation, inspected September 15, 2026. Documentation inspection only; not a compiled integration test. https://leanprover-community.github.io/mathlib4_docs/Mathlib/LinearAlgebra/Matrix/ToLin.html.
- [12] Mathlib contributors, *Mathlib.Algebra.Homology.Homotopy*, official API documentation, inspected September 15, 2026. Includes `Homotopy`, `Homotopy.homologyMap_eq`, and `HomotopyEquiv.toHomologyIso`. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Algebra/Homology/Homotopy.html.
- [13] E. Sköldbberg, “Morse theory from an algebraic viewpoint,” *Transactions of the American Mathematical Society* **358**(1) (2006), 115–129. DOI: 10.1090/S0002-9947-05-04079-1. Institutional publication record.